Predicting Loan Default on Lending Club Data: Application-Time Scoring with Out-of-Time Validation

Phung Thi Thu Trang

DS66B, School of Technology, National Economics University

November 5, 2026

Outline

1. Problem Statement
2. Existing Methodologies
3. Proposed Method
4. Experiments
5. Conclusion

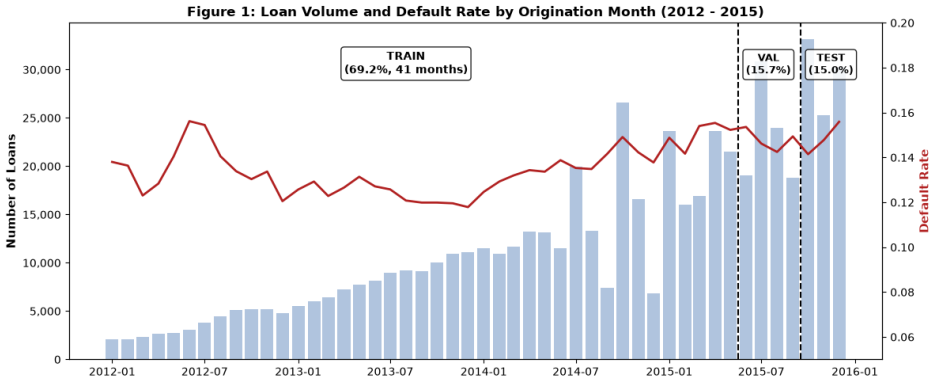


Figure: Loans by issue month and the train / validation / test split

Problem Statement

Challenge: Rank loan applicants by default risk using only information known when the loan is approved

Key Issues:

- **Data leakage**: many of the 151 columns are created after the loan starts
- **Temporal drift**: default rate rises from 13.7% (train) to 14.8% (test)
- **Right-censoring**: recent loans have no final outcome yet
- **Incumbent model**: Lending Club's own `sub_grade` (A1–G5) is already a risk score

Solution: A leakage-controlled, out-of-time pipeline compared against the `sub_grade` baseline

Task Definition and Metrics

Binary classification: will a new loan be charged off?

- **Input**: data known at approval (amount, income, job, credit history)
- **Model**: scoring model trained on old loans
- **Output**: $P(\text{default}) \in [0, 1]$, used to rank risk
- **Label**: 0 = Fully Paid, 1 = Charged Off

Main rule: no data created after approval

Metrics (ranking quality):

- **ROC-AUC**: chance a bad loan scores above a good loan
- **Gini** $= 2 \times \text{AUC} - 1$: standard score in credit risk
- **KS**: max gap between bad and good cumulative rates, $\max(\text{TPR} - \text{FPR})$

No probability cut-off is needed: all three measure ranking.

Related Work Overview

Studies on Lending Club data:

- Serrano-Cinca et al.: grade is the most predictive factor; logistic regression [1]
- Malekipirbazari & Aksakalli: random forests beat logistic regression, FICO and LC grades [2]
- Limitation: different data, filters and splits; not comparable

Evaluation over time:

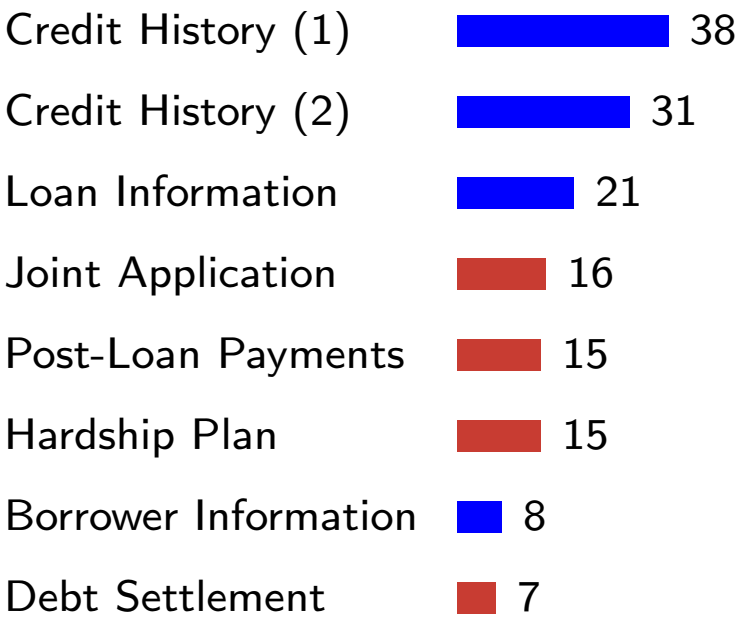
- Qian et al.: mixing periods leaks the future and inflates results [3]
- Open-source projects: only some use out-of-time splits and the platform grade as benchmark [4, 5]
- Limitation: leakage and time control rarely verified

Our Contribution: A reproducible study with an out-of-time split, a `sub_grade` baseline with bootstrap intervals, and an ablation without `sub_grade`

Data Dictionary

151 columns in 8 groups, each tagged: known at application?

Columns per group:



Red = groups only filled after the loan starts

Timing tag:

- **103** Yes: known at application
- **41** No: after loan or ID
- **7** Uncertain: timing not clear

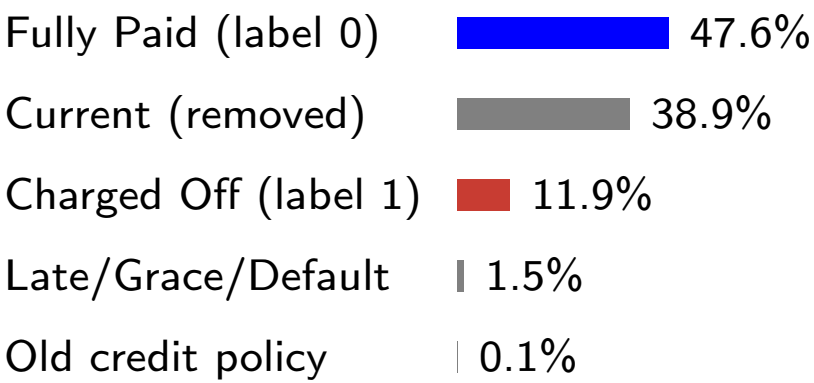
151 / 151: dictionary matches the raw file header

Data Ingestion and Target Definition

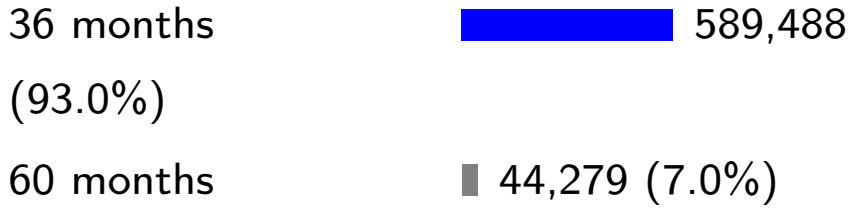
Only loans with a final result get a label

- **2,260,701** rows × 151 columns; **1,345,310** labelled rows
- Default rate among labelled loans: **20.0%**

Loan status (% of all rows):



Loan term (matured, from 2012):

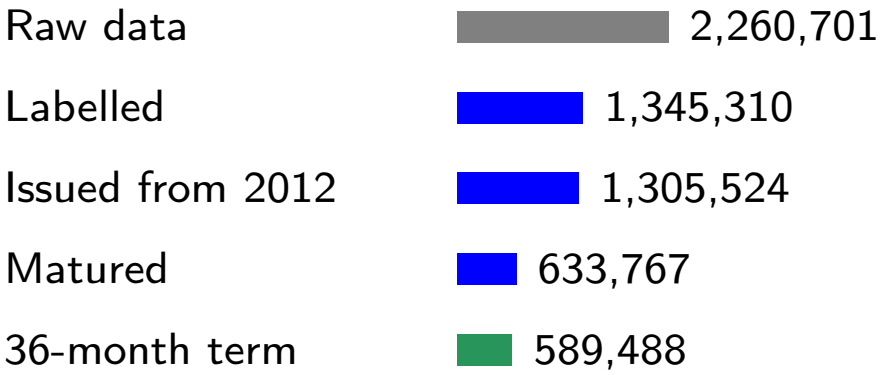


Keep 36-month loans only, so all 3 sets share one term

Data Filtering and Out-of-Time Split

Train on old loans, test on newer loans: 70 / 15 / 15 by date

Filter funnel (rows):



Three sets:

- **Train** 408,188 (69.2%): 2012-01 $\rightarrow$ 2015-05, default 13.7%
- **Val** 92,631: 2015-06 $\rightarrow$ 09, default 14.7%
- **Test** 88,669: 2015-10 $\rightarrow$ 12, default 14.8%

- **Matured**: issue month + term $\leq$ 2018-12 (99.6% of matured loans are labelled vs 42.4%)
- No month in two sets; default rate drifts up +1.1 points

Data Cleaning and Leakage Prevention

Each rule checks one row; the same rule runs on train, val and test

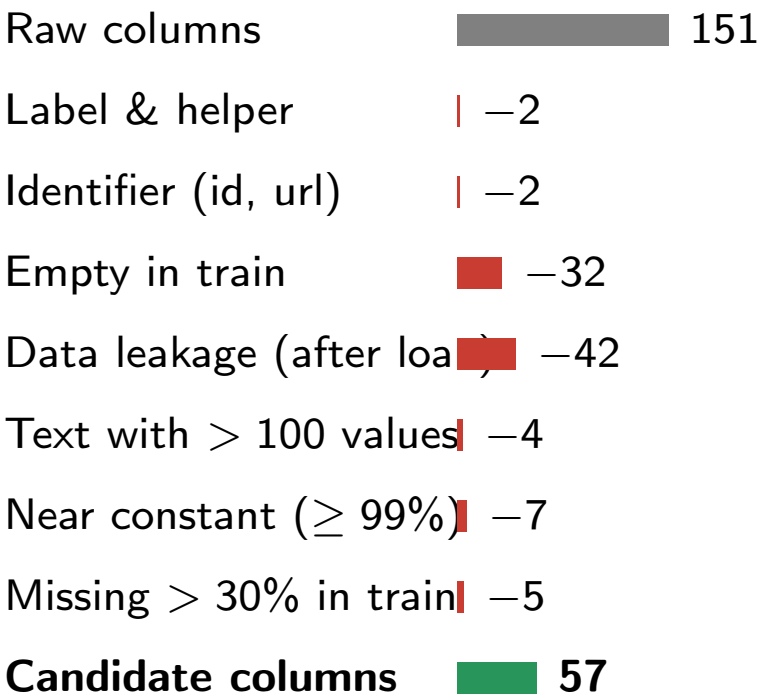
#	Logic rule (check)	Why	Handling
1	`annual_inc` ≤ 0	Income must be positive	Set to missing
2	`loan_amnt` ≤ 0	Amount must be positive	Set to missing
3	`dti` < 0 or > 100	DTI must be 0–100	Set to missing
4	`revol_util` > 100	Possible; extremes are noise	Cap at 102 (train p9
5	`fico_low` $>$ `fico_high`	Low must not pass high	Set both to missing
6	`earliest_cr_line` $>$ `issue_d`	Credit before the loan	Set to missing

Anti-leakage rules:

1. Split first, transform later
2. Median, mean, std, caps: **train only**
3. Pipeline `fit` on train; val and test only `transform`

Feature Selection: 151 → 57

Each column goes through filters in a fixed order; first match removes it



Leakage check (single-column AUC, train):

- Best legitimate: `sub_grade` 0.6565
- `last_fico_range_low` 0.9074
- `recoveries` 0.9054
- `total_rec_prncp` 0.8280

Post-loan columns are far too strong

Feature Engineering and Preprocessing: 57 → 153

New features, remove duplicates, then impute, scale and one-hot in one pipeline

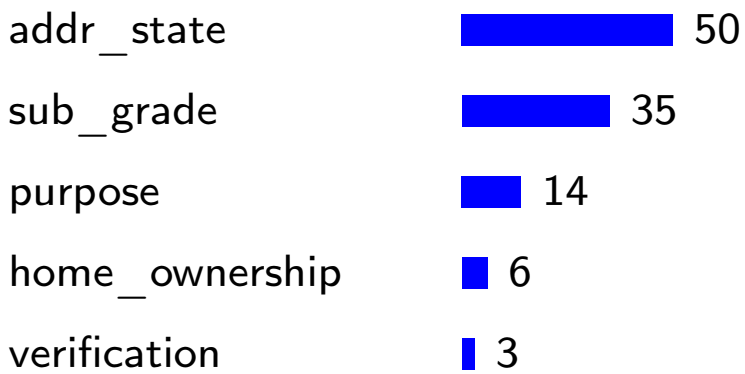
57	→	+1	→	+6	→	−14	→	50
candidates		issue_date (anchor)		new features		duplicates & high corr.		45 num + 5 cat

→ **153** final matrix

6 new features:

- `fico_avg`: mean of FICO range
- `credit_history_years`: years of credit
- `loan_to_income`: loan / income
- `log_annual_inc`, `log_revol_bal`: fix skew
- `emp_length_years`: text → number

One-hot columns (108 total):



45 numeric: median impute + StandardScaler

Experimental Setup

Dataset: Lending Club accepted loans (Kaggle [6])

- Issued 2007–2018, filtered to matured 36-month loans from 2012
- Out-of-time split by issue date

Table: Summary of the three sets after filtering

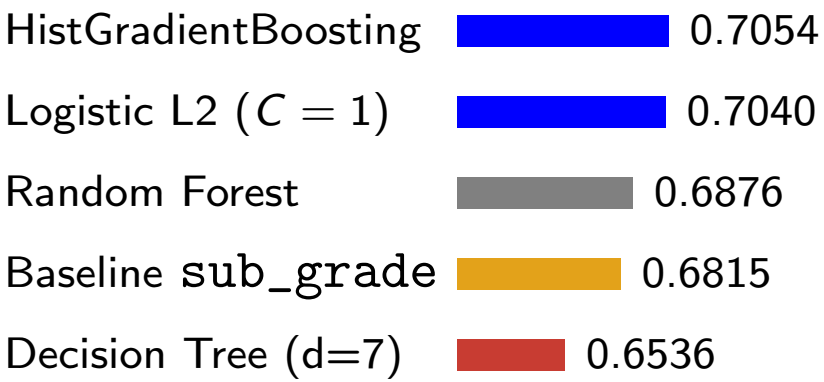
Set	Loans	Period	Default rate
Train	408,188	2012-01 – 2015-05	13.7%
Validation	92,631	2015-06 – 2015-09	14.7%
Test	88,669	2015-10 – 2015-12	14.8%

Models: Baseline `sub_grade`, Logistic L2, Decision Tree, Random Forest, HistGradientBoosting
Tuning: `GridSearchCV` with `TimeSeriesSplit(3)` inside train
Metrics: ROC-AUC, Gini, KS; paired bootstrap ($B = 1{,}000$) for 95% CI of ΔAUC

Model Training and Validation

5 models tuned with TimeSeriesSplit; compared on the validation set

Validation ROC-AUC (axis starts at 0.60)



Overfitting (train − val AUC):

Random Forest +0.018; HistGB +0.004; Logistic −0.017

Bootstrap 95% CI of ΔAUC vs baseline:

Model	95% CI
HistGB	[+0.021, +0.027]
Logistic L2	[+0.020, +0.025]
Random Forest	[+0.002, +0.010]
Decision Tree	[−0.033, −0.023]

Validation Curves and Metrics

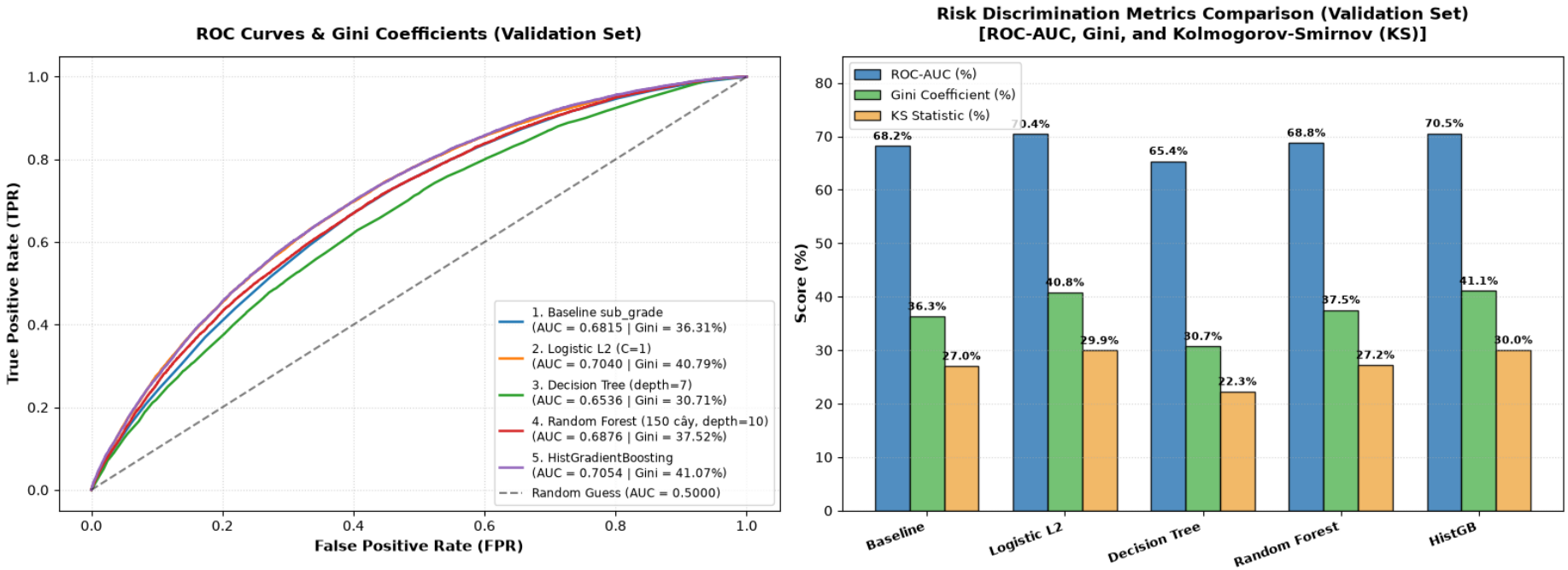


Figure: ROC curves, Gini, AUC and KS of the five models on the validation set (92,631 loans)

Tuned settings: Logistic $C = 1$; Tree depth 7; Random Forest 150 trees, depth 10; HistGB 150 iterations, depth 10, learning rate 0.05

Out-of-Time Test Results and Decision

Test set used once: 88,669 loans, 2015-10 to 2015-12

Table: Out-of-time test performance

Model	ROC-AUC	Gini	KS
Baseline `sub_grade`	0.6818	36.4%	26.5%
Logistic L2 (chosen)	**0.7039**	40.8%	29.9%
HistGradientBoosting	0.7055	41.1%	30.0%

HistGB − Logistic (95% CI): $[-0.0002, +0.0031]$ contains 0, no real difference

Decision: Logistic L2

- Beats baseline: +0.022 AUC
- Same level as HistGB; no overfitting, fast to train
- Each coefficient is easy to explain

Experiment 1: Ablation without `sub_grade`

Remove Lending Club's own grade: 153 → 118 features. Is the model still better?

Test ROC-AUC (axis from 0.66)

L2 (+sub_grade) 0.7039

L2 (no sub_grade) 0.6879

Baseline `sub_grade` 0.6818

- **+0.0075** no sub_grade vs baseline, CI [+0.003, +0.012]
- **+0.0150** value added by sub_grade, CI [+0.013, +0.017]

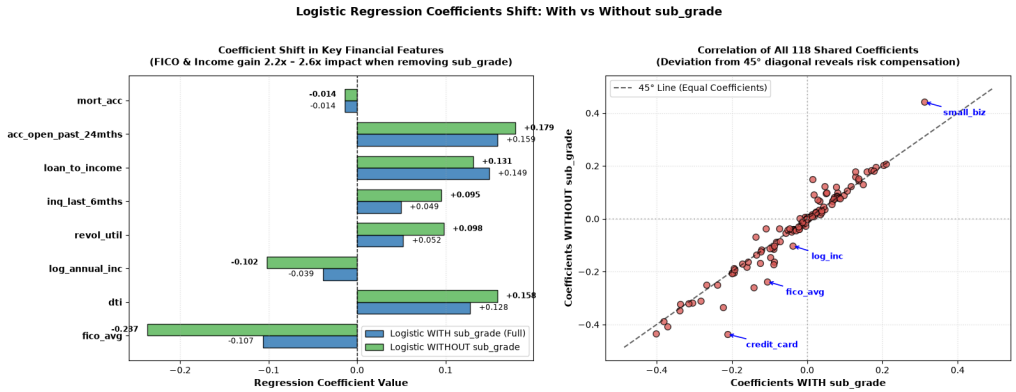


Figure: Coefficient shift when `sub_grade` is removed

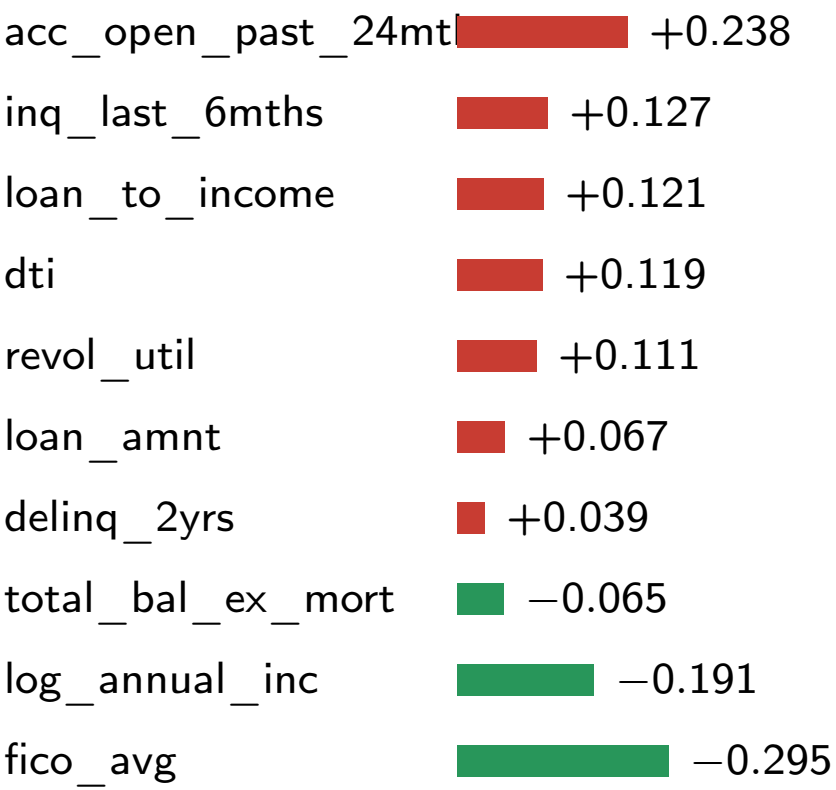
`sub_grade` absorbs the risk signal; without it, weight moves back to FICO, income and credit use (1.2×–2.6×)

Experiment 2: Compact Model and Credit Scorecard

12 core features, no `sub_grade`, turned into a 300–850 style score

- **153** $\rightarrow$ **30** columns (-80%); test ROC-AUC **0.673** (Gini 34.6%); score range 494–757 (validation)

Weights of 10 numeric features:



Score scaling:

- Base score 600 at odds 5 : 1
- PDO: 20 points
- Example cut-off: 615
- Higher score = lower risk

2 categorical features:

`home_ownership`, `purpose`

Fewer inputs: cheaper data collection, easy to explain to a loan officer

Experiment 3: Score Distribution, Good vs Bad Borrowers

Compact 12-feature scorecard, test set, 88,669 loans

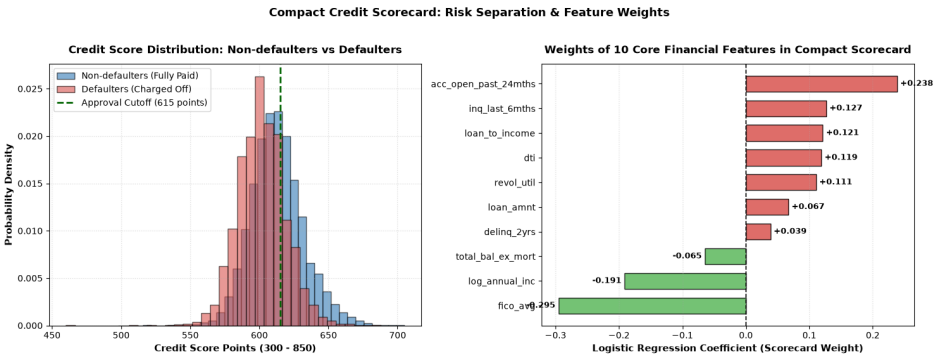


Figure: Score distribution (left) and feature weights (right)

Default rate by band (avg 14.8%)

$<$ 590 Reject	30.2%
590–614 Review	17.3%
615–639 Approve	8.7%
$\geq$ 640 Approve	3.3%

9$\times$ default gap: lowest vs highest band

25.1% test KS

Conclusions

Summary:

- Logistic L2 and HistGB beat the `sub_grade` baseline out of time (test AUC 0.7039 and 0.7055 vs 0.6818) and cannot be told apart
- Without `sub_grade`, Logistic still beats the baseline (0.6879), and FICO and income coefficients about double
- A compact 12-field scorecard is below the baseline in AUC (0.6730) but separates risk bands from 30.2% to 3.3% default

Key Technical Contributions:

1. **Leakage control**: application-time whitelist checked by single-column AUC
2. **Out-of-time protocol**: matured 36-month loans, chronological 70/15/15 split, pipeline fitted on train only
3. **Honest evaluation**: bootstrap intervals, single test use, negative results reported

Limitations and Future Work

Limitations:

- 36-month loans of 2012–2015; 3-month test window
- Random Forest and HistGB grids hit their edges
- Bootstrap CI covers validation sampling only
- Val AUC $>$ train AUC for the baseline and Logistic: not explained
- No calibration or cost analysis; scorecard constants are conventions

Research Directions:

1. Wider hyperparameter search
2. Rolling-origin evaluation and dataset-shift tests [3]
3. Missing-indicator treatment of `mths_since_...` columns
4. Calibration and cost-sensitive thresholds
5. 60-month loans via survival analysis

References I

C. Serrano-Cinca, B. Gutiérrez-Nieto, and L. López-Palacios, "Determinants of default in P2P lending," *PLoS ONE*, vol. 10, no. 10, p. e0139427, 2015.

M. Malekipirbazari and V. Aksakalli, "Risk assessment in social lending via random forests," *Expert Systems with Applications*, vol. 42, pp. 4621–4631, 2015.

H. Qian, B. Wang, P. Ma, L. Peng, S. Gao, and Y. Song, "Managing dataset shift by adversarial validation for credit scoring," arXiv preprint arXiv:2112.10078, 2021.

Arturo-GA, "lendingclub-default-risk," GitHub repository, https://github.com/Arturo-GA/lendingclub-default-risk, description based on the repository's README page; code and reported results were not reproduced.

vaibhavkev, "credit-risk," GitHub repository, https://github.com/vaibhavkev/credit-risk, description based on the repository's README page; code and reported results were not reproduced.

N. George, "Lending club loan data (accepted and rejected loans, 2007–2018)," Kaggle dataset (user `wordsforthewise`), https://www.kaggle.com/datasets/wordsforthewise/lending-club, accessed 2026.

References II


N. Siddiqi, *Intelligent Credit Scoring: Building and Implementing Better Credit Risk Scorecards*, 2nd ed., ser. Wiley and SAS Business Series. Hoboken, NJ: John Wiley & Sons, 2017.


D. J. Hand and W. E. Henley, "Statistical classification methods in consumer credit scoring: a review," *Journal of the Royal Statistical Society: Series A*, vol. 160, no. 3, pp. 523–541, 1997.


F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.


J. H. Friedman, "Greedy function approximation: a gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.


L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

References III


G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: a highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 3146–3154.


B. Efron, "Bootstrap methods: another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.


T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.


F. J. Massey, "The Kolmogorov–Smirnov test for goodness of fit," *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.